

# Worksheet — 1.1 Processors, Input, Output & Storage — ANSWER SHEET

---

*Separate answer sheet / mark scheme — keep for marking.*

## Answer key

---

**1. Key-term match** 1–C, 2–G, 3–A, 4–F, 5–B, 6–E, 7–D, 8–H.

**2. Fill in the blanks (FDE)** ...copied from the **PC** into the **MAR**. A read signal is sent on the **control** bus, and the instruction returns along the **data** bus into the **MDR**. The processor then **increments** the PC... copied from the **MDR** into the **CIR**. Decode: splits the instruction into the **opcode** (the operation) and the **operand** (the data or **address**). Execute: a calculation is performed by the **ALU**, result stored in the accumulator.

**3. State the register** a) PC b) MAR c) MDR d) CIR e) ACC

**4. Put the FDE steps in order** 1 — The address in the PC is copied into the MAR. 2 — Read signal placed on the control bus; instruction returns via the data bus into the MDR. 3 — The PC is incremented ( $PC := PC + 1$ ). 4 — The contents of the MDR are copied into the CIR. *(Steps 3 and 4 may be quoted either way round; the key points are that the PC must be incremented and the MDR must be copied into the CIR.)*

**5. Short answer — performance & architectures** a) Any two, e.g.: - **Clock speed** — pulses per second synchronise the CPU; a higher clock speed means more FDE cycles per second (limited by heat/power). - **Cores** — each core has its own FDE cycle, so multiple instructions can be processed simultaneously. - **Cache** — small, very fast memory on/near the CPU holding frequent/recent data, reducing slow fetches from RAM. - **Pipelining** — overlaps fetch/decode/execute of consecutive instructions, increasing throughput. b) Extra cores only help **if the task can be split into parallel parts and the software is written to use multiple cores**; inherently sequential tasks gain little. c) **Von Neumann** uses one shared memory and bus for both data and instructions (simpler/cheaper but causes a bottleneck because it cannot fetch an instruction and read/write data at the same time); **Harvard** uses separate memories/buses for data and instructions, allowing simultaneous access. Use: Von Neumann — general-purpose PCs/laptops; Harvard — embedded systems / microcontrollers / DSPs. d) Any two, e.g.: CISC has many complex, variable-length, multi-cycle instructions with complexity in hardware; RISC has few simple fixed-length instructions aiming for one per cycle with complexity in the compiler. RISC pipelines more easily and uses less power (good for battery devices/ARM); CISC is typical of x86 desktops. e) A GPU has thousands of small cores that apply the **same operation to many data items in parallel (data parallelism)**, which suits pixels/vertices and large ML matrices. Branch-heavy, dependency-laden or inherently sequential work cannot be parallelised this way (each step needs the previous result), so a CPU handles it better.

**6. Storage device selection table** (example answers)

| Type             | How it works                                                                           | One pro                               | One con                              | Example use                                         |
|------------------|----------------------------------------------------------------------------------------|---------------------------------------|--------------------------------------|-----------------------------------------------------|
| <b>Magnetic</b>  | Magnetised patterns/polarity on a spinning platter or tape, read/written by a R/W head | Huge capacity, low cost per GB        | Slow (moving parts), fragile         | Bulk/backup storage; tape archive                   |
| <b>Flash/SSD</b> | EEPROM cells trap electrons; no moving parts                                           | Fast, silent, low power, durable      | Costlier per GB, finite write cycles | Laptop main drive; USB stick; SD card               |
| <b>Optical</b>   | Laser detects pits and lands via reflected light                                       | Very cheap, portable, long shelf life | Low capacity, slow, easily scratched | Distributing films/software; long-term disc archive |

**7. Challenge box** (i) Virtual memory uses **secondary storage (a swap/paging file) as extra RAM** when physical RAM is full; pages are moved out to disk and back as needed. It is being used because the open applications need more memory than the installed RAM provides. (ii) The problem is **disk thrashing** — excessive paging. It is slow because **secondary storage (disk) is far slower than RAM**, so constantly swapping pages in and out wastes most of the CPU's time waiting. (iii) e.g. **Install more RAM** so fewer pages need swapping to disk (less/no paging), or **fit an SSD** so any paging that does occur is much faster than on an HDD.